InPlace AI: Implementation Plan

Frontend Technologies

Technology: React with TypeScript
Rationale: React provides a component-based architecture ideal for building interactive user interfaces. TypeScript adds type safety, reducing bugs and improving code maintainability. The large ecosystem and community support ensure long-term viability.

UI Framework: Tailwind CSS with shadcn/ui component library
Rationale: Tailwind enables rapid, consistent styling while shadcn/ui provides accessible, pre-built components that can be customized. This combination accelerates development while maintaining design flexibility and accessibility compliance.

State Management: React Query
Rationale: React Query handles server state, caching, and data synchronization efficiently.

Form Management: React Hook Form with Zod validation
Rationale: React Hook Form minimizes re-renders for better performance, especially important for file upload forms. Zod provides runtime type validation, ensuring data integrity before submission.

File Upload: React Dropzone with progress tracking
Rationale: Provides an accessible drag-and-drop interface with visual feedback, essential for senior-friendly photo uploads.

Backend Technologies

Framework: Next.js (App Router)
Rationale: Next.js offers both frontend and backend capabilities in one framework, simplifying deployment. Server-side rendering improves performance and SEO. API routes provide a straightforward backend solution without separate infrastructure.

API Architecture: RESTful API with Next.js API Routes
Rationale: REST is well-understood, widely supported, and sufficient for our needs. Next.js API routes keep the stack unified and reduce complexity.

Authentication: NextAuth.js
Rationale: Provides secure authentication with support for multiple providers, session management, and role-based access control. Integrates seamlessly with Next.js.

File Processing: Sharp (image optimization) + PDF-lib (PDF generation)
Rationale: Sharp handles image resizing and optimization efficiently. PDF-lib generates grant documents programmatically without external dependencies.

Background Jobs & Workflow Orchestration: Redis-backed queue
Rationale: Required for asynchronous tasks that should not block the user experience, including virus scanning, image optimization, AI estimation, PDF generation, BuilderTrend transfer retries, and email delivery.

Database & Storage

Primary Database: PostgreSQL
Rationale: PostgreSQL is a robust, open-source relational database with excellent JSON support for flexible data storage. It handles complex queries efficiently and scales well for our projected growth.

ORM: Prisma
Rationale: Prisma provides type-safe database access with an intuitive schema definition. Auto-generated migrations simplify database versioning. Strong TypeScript integration catches errors at compile time.

File Storage: AWS S3 (or compatible service like DigitalOcean Spaces)
Rationale: Object storage is cost-effective for photos and documents. S3 offers durability, scalability, and easy integration with CDNs for fast content delivery. Supports automatic lifecycle policies for data retention compliance.

AI/ML Infrastructure

Computer Vision: OpenAI API (vision-capable model, e.g., GPT-4o) with optional fallback to Google Cloud Vision API
Rationale: Supports photo-based analysis for scope inference without custom model training. Keeping a fallback option reduces vendor risk and supports LandSeed compliance constraints if required.

Third-Party Integrations

BuilderTrend Integration

Email Service: SendGrid or Amazon SES
Rationale: Both offer reliable transactional email delivery with template management, delivery tracking, and high deliverability rates. SendGrid has a more user-friendly interface while SES is more cost-effective at scale.

DevOps & Infrastructure

Hosting Platform: Vercel (for Next.js app) + AWS/DigitalOcean (for PostgreSQL)
Rationale: Vercel is optimized for Next.js with automatic deployments, edge functions, and excellent performance. A separate database hosting provides more control over scaling and backups.

Alternative: AWS with Amplify or full self-hosted on DigitalOcean
Rationale: Provides more control and potentially lower costs at scale, though requires more DevOps expertise.

CI/CD: GitHub Actions
Rationale: Free for open source projects, integrates directly with GitHub repositories, supports automated testing and deployment pipelines.

Monitoring & Analytics: Sentry (error tracking) + Vercel Analytics (performance)
Rationale: Sentry captures errors with detailed context for quick debugging. Vercel Analytics provides real-time performance metrics to catch regressions and guide optimization.

Logging: Better Stack (formerly Logtail) or CloudWatch
Rationale: Centralized logging with search capabilities essential for debugging production issues and maintaining audit trails.

Testing & Quality Assurance

Unit Testing: Jest + React Testing Library
Rationale: Jest is the industry standard for JavaScript testing. React Testing Library encourages testing user behavior rather than implementation details.

End-to-End Testing: Playwright
Rationale: Playwright supports multiple browsers, provides reliable test execution, and includes built-in debugging tools. Better than Cypress for our needs due to superior handling of file uploads and multi-tab scenarios.

API Testing: Jest (automated tests)
Rationale: Jest handles automated API testing in CI/CD pipeline.

Accessibility Testing: axe-core + manual testing
Rationale: Automated accessibility testing catches common issues. Manual testing with screen readers ensures real-world usability for users with disabilities.

Project Management & Communication

Project Management: Jira or Linear
Rationale: Jira is industry-standard with robust features but can be complex. Linear offers a more modern, streamlined experience. Either supports agile workflows, sprint planning, and progress tracking.

Communication: Slack (as suggested by LandSeed)
Rationale: Real-time communication for quick questions and updates. Integrates with GitHub and project management tools for automated notifications.

Documentation: Notion (project docs) + Swagger/OpenAPI (API docs)
Rationale: Notion provides collaborative documentation space. Swagger auto-generates interactive API documentation from code annotations.

Version Control: Git with GitHub
Rationale: Industry standard, supports collaborative development, code review workflows, and integrates with all our development tools.

Security Tools

Dependency Scanning: Snyk or GitHub Dependabot
Rationale: Automatically identifies vulnerabilities in dependencies and suggests updates.

Secret Management: Environment variables with Vercel Env + AWS Secrets Manager (production)
Rationale: Keeps sensitive credentials out of code. Secrets Manager provides encryption at rest and rotation capabilities.

SSL/TLS: Let's Encrypt (via Vercel) or AWS Certificate Manager
Rationale: Free, automated SSL certificate management ensures encrypted connections.